Topics Covered in Lecture

- What is software and software engineering?
- How software engineering is different from computer science and system engineering.
- Key FAQs related to software engineering.
- Importance of software engineering in today's world.

❓ Frequently Asked Questions (FAQs)

1. What is Software?

Software is more than just code. It includes:

- **Program**: The actual code.
- **Data**: The information the program works on.
- **Documentation**: Manuals, design diagrams, user guides, etc.

2. Types of Software

- **Custom Software**: Made for a specific customer.
- **Generic Software**: Sold to many users (e.g., MS Word).
- **Embedded Software**: Built into hardware (e.g., microwave software).
- **Real-Time Software**: Must respond immediately (e.g., airplane control).
- **Data Processing Software**: Business-related (e.g., billing systems).

🛒 Software Products

- **Generic Products**: Same product for many users (e.g., Photoshop).
- **Customized Products**: Built for one user's specific needs (e.g., traffic control system).

Ownership of Specifications:

- **Generic**: Developer owns and updates.

- **Customized**: Customer decides what features are needed.

🧩 Software Applications

1. **System Software** – like compilers and file managers.
2. **Application Software** – like word processors.
3. **Scientific/Engineering Software** – used in research (e.g., NASA tools).
4. **Embedded Software** – inside devices.
5. **Product Line Software** – made for a group of users (e.g., Office Suite).
6. **Web Apps** – internet-based software (e.g., Gmail).
7. **AI Software** – like expert systems and pattern recognition.

🌐 Changing Trends in Software

Modern software is growing in these four areas:

- **Web Applications**
- **Mobile Applications**
- **Cloud Computing**
- **Product Line Software**

👷 What is Software Engineering?

✅ Definition:

Software engineering is the use of engineering techniques to develop software reliably, efficiently, and within budget.

🔍 Key Ideas:

- Not just coding: it also includes planning, designing, testing, managing, and maintaining software.
- Must be **systematic**, **organized**, and based on **engineering principles**.

🧱 IEEE Definition:

A **disciplined**, **measurable** method for software development, operation, and maintenance.

🔬 Software Engineering vs Computer Science

Computer Science	Software Engineering
Focuses on **theories** & fundamentals	Focuses on **real-world software building**
Academic	Practical
Example: Algorithms, complexity	Example: Making a working website or app

---

🖥️ Software Engineering vs System Engineering

System Engineering	Software Engineering
Covers both hardware and software	Focuses only on software
Deals with entire systems (e.g., full computer systems)	Deals with software part of the system
Includes architecture, integration, and deployment	Includes design, coding, testing

---

🧱 Software Engineering as a Layered Technology

1. **Quality Focus** – The goal is to build high-quality software.
2. **Process Layer** – Provides steps and guidelines to manage software development (like a roadmap).
3. **Methods** – Techniques like designing, coding, testing, communication, etc.
4. **Tools** – Software tools to support the above (e.g., Visual Studio, Git, CASE tools).

🧰 Software Engineering Framework

This framework helps in managing the software development process efficiently:

- **Quality Focus**: Ensures software meets high standards.
- **Process**: Organizes activities from start to finish.
- **Methods**: Step-by-step ways to build software.
- **Tools**: Help automate or assist in development tasks.

🏆 Importance of Software Engineering

- Society depends on software for daily tasks — banking, transport, healthcare, etc.
- Helps build **reliable**, **economical**, and **maintainable** systems.
- Saves time and money in the long run compared to unplanned coding.
- Makes software easier to change, test, and reuse.

🌟 What is a Software Process?

A **software process** is a **systematic approach** to developing or evolving software.
It includes **a set of activities** such as:

1. **Specification** – What should the system do?
2. **Development** – Designing and programming the system.
3. **Validation** – Ensuring the software meets customer needs.
4. **Evolution** – Updating software for changing needs.

🧩 Simplified Software Process Flow

- **Feasibility & Planning**
- **Requirements**
- **Design**
- **Implementation**
- **Operation & Maintenance**

🔄 What is a Software Process Model?

A **simplified representation** of a software process from a specific view:

- **Workflow**: What is done and when?
- **Data flow**: What information flows where?
- **Role/action**: Who does what?

Common Process Models:

- **Waterfall**
- **Iterative development**
- **Component-based engineering**

💰 Software Engineering Costs

- ~60%: **Development**
- ~40%: **Testing**
- **Custom software**: Evolution may cost more than development.
- Costs vary by:
    - **System type**
    - **Performance & reliability**
    - **Development model used**

📊 Activity Cost Distribution

- **Waterfall**: Front-loaded with planning and specification.
- **Iterative**: Continuous cycles of design, development, and testing.
- **Component-based**: Heavy focus on integration and reuse.

🛠️ What are Software Engineering Methods?

Structured approaches to software development involving:

- **Models** – Diagrams and representations
- **Rules** – Constraints on system models
- **Recommendations** – Best design practices
- **Process guidance** – Steps to follow

💻 What is CASE (Computer-Aided Software Engineering)?

Tools that **automate** software development tasks:

- **Upper-CASE** – Supports early stages (requirements, design)
- **Lower-CASE** – Supports later stages (coding, testing)

🌈 Attributes of Good Software

1. **Maintainability** – Can evolve with user needs.
2. **Dependability** – Reliable and secure.
3. **Efficiency** – Uses system resources well.
4. **Acceptability** – Easy to use and fits with other systems.

⚠️ Key Challenges in Software Engineering

- **Heterogeneity** – Different platforms and environments.
- **Delivery** – Faster production and deployment.
- **Trust** – Prove the software is reliable.
- **Legacy systems** – Maintain and upgrade existing systems.

👥 Who are Stakeholders?

Anyone with an interest in the success of the project:

- **Users** – Use the software.

- **Customers** – Pay for the software.
- **Developers** – Build the software.
- **Managers** – Oversee the process.

🚧 Difficulties and Risks

- High complexity
- Changing requirements
- Limited skills
- Political factors
- Tech uncertainties

🔑 Software Engineering Fundamentals

- Use a **managed and understood process**.
- Ensure **dependability** and **performance**.
- **Understand and manage** requirements.
- **Reuse** existing components where possible.

🌐 Web-Based Software Engineering

- Web is now a **platform** (not just for static pages).
- Focus on **software reuse** and **component integration**.
- Development is **incremental**, not fully spec'd upfront.
- Use of **AJAX**, **JavaScript**, **web forms** is common.
- Same fundamental principles apply as for traditional software.

❓ Frequently Asked Questions (FAQs)

Question	Answer
What is software?	Programs + documentation.
Good software attributes?	Maintainable, dependable, efficient, acceptable.
What is software engineering?	The discipline of software production.
Main activities?	Specification, Development, Validation, Evolution.
Difference from CS?	CS = theory; SE = practical software creation.
Difference from Systems Engineering?	SE = software only; Systems Eng = software + hardware.
Web impact?	Enables distributed, service-based systems & software reuse.
Major challenges?	Diversity, delivery speed, trust, legacy systems.
Best method?	Depends on the system (e.g. games = prototypes, safety = formal specs).
Cost distribution?	~60% dev, ~40% testing; evolution can cost more for custom software.

🧠 **Keywords for Revision:**

- Software Process
- Specification
- Evolution
- Process Models
- CASE Tools
- Maintainability
- Heterogeneity
- Stakeholders
- Web-based Engineering

What is a Software Process?

A **software process** is a set of structured activities to **develop or evolve software**, typically including:

1. **Specification** – What the system should do.
2. **Development** – Designing and coding the software.
3. **Validation** – Making sure the software meets customer needs.
4. **Evolution** – Updating software as needs change.

🔹 **Steps in Software Development**

1. **Feasibility and Planning**
2. **Requirements**
3. **Design**
4. **Implementation (Coding)**
5. **Acceptance and Release**
6. **Operation and Maintenance**

Each of these may be revisited in different models.

🔹 **Software Process Models**

1. Build and Fix

- Just code and fix errors.
- No clear requirements or design → Unreliable.
- Not recommended for serious projects.

2. Sequential Models

- **Waterfall Model**:
    - Complete one phase before the next.
    - Works best when requirements are clear.

- **V-Model**:
    - Extension of waterfall.
    - Emphasizes testing at each development stage.

3. Iterative Models

- Build a rough version quickly → Improve through cycles.
- Encourages early feedback.

4. Incremental Models

- Develop and release the system in small chunks.
- Each increment adds functionality.
- Basis of Agile methods.

5. Evolutionary Development

- **Prototyping**:
    - Create a working model early.
    - Helps clarify requirements.
- **Spiral Model**:
    - Combines iterative nature with risk analysis.

🔹 **Agile Process Models**

- Emphasize flexibility, collaboration, and customer feedback.
- Examples:
    - **RUP** (Rational Unified Process)
    - **RAD** (Rapid Application Development)
    - **Scrum**, **Extreme Programming** (XP)

🔹 **Key Activities in All Models**

- **Project Management**

- **Quality Assurance**
- **Configuration Management**
- **Requirement Engineering**
- **Design, Coding, Testing**
- **Installation, Maintenance, Integration**

🔹 **Quality Control Steps**

- Validate requirements and design
- Usability and program testing
- Acceptance testing
- Bug fixing and maintenance

🔹 **Summary of Process Flow**

Basic Phases:

1. Feasibility Study
2. Requirements Gathering
3. System & Program Design
4. Coding & Unit Testing
5. System Testing & Acceptance
6. Operation, Maintenance, and Evolution

Each model uses these steps differently based on how flexible or strict it is.

Software Process Models Summary Notes

---

1. Heavyweight vs. Lightweight Software Development

- **Heavyweight Process**:
    - Development steps are slow and systematic.

    - Focus on completing each process step thoroughly with minimal changes.
    - Example: Modified Waterfall Model.
- **Lightweight Process**:
    - Software is developed and released in small increments.
    - Plans evolve incrementally based on experience.
    - Changes are expected as development progresses.
    - Example: Agile Software Development.

---

2. Deliverables

- **Definition**: A deliverable is a work product delivered to the client at the end of each development phase.
- **Heavyweight Process**: Each phase produces deliverables (often documentation), e.g., requirements specification.
- **Lightweight Process**: Deliverables are working software increments with minimal documentation.

---

3. Software Life-Cycle Models (SDLC)

- **Life-Cycle Model**: A framework describing activities at each stage of a software development project.
- **Generic Process Models**:
    - Waterfall
    - Evolutionary Development
    - Component-based Software Engineering
    - Iterative Development

---

4. Types of Software Process Models

- **Sequential Models**:

    - Waterfall Model
    - Phases: Requirements, Design, Implementation, Testing, Installation, Maintenance.
- **Incremental Process Models**:
    - Combines waterfall elements in an iterative fashion.
    - Functionality is delivered in stages.
- **Evolutionary Process Models**:
    - Phases like specification, development, and validation are interleaved.
- **Prototyping**:
    - Early versions of software are built, refined, and finalized.
- **Spiral Model**:
    - Focuses on risk assessment and iterative refinement.

---

5. Waterfall Model

- **Phases**:
    - Requirements Analysis
    - Design
    - Implementation
    - Testing
    - Installation
    - Maintenance
- **Advantages**:
    - Easy to understand.
    - Clear structure, good for inexperienced staff.
    - Milestones are well-defined.
    - Good for management control.

- **Disadvantages**:
    - All requirements must be known upfront.
    - Difficult to accommodate changes during the process.
    - Limited flexibility, as integration happens only at the end.
- **Best Use**: When requirements are well-known and stable (e.g., new version of an existing product, porting an existing product).

---

6. Modified Waterfall Model

- An adaptation of the traditional Waterfall model to allow for limited flexibility.
- Useful when some requirements may change but a structured approach is still necessary.

---

7. Incremental Process Model

- **Characteristics**:
    - Software is developed in small, functional releases (increments).
    - Each increment builds on the previous one.
    - Core product is developed first, followed by additional features in later releases.
- **Strengths**:
    - High-risk features are developed first.
    - Customer can evaluate each increment.
    - Reduces the cost of the initial release.
    - Allows flexibility and easier bug detection during each iteration.
- **Weaknesses**:
    - Requires good planning and well-defined modules early on.
    - Total system cost can be higher than in the waterfall model.

- **Best Use**: When there's a need for early delivery, feedback from users is important, or when the project involves new technologies.

---

8. When to Use Each Model

- **Waterfall Model**:
    - Well-defined requirements.
    - Stable design needed from the beginning.
    - Projects with minimal expected changes.
- **Incremental Model**:
    - Need for early realization of basic functionalities.
    - Projects with evolving requirements or new technology.
    - Risk management and timely delivery are priorities.

Process Iteration

- **Iterative Development** is crucial because system requirements often change during the course of a project, especially for large systems. Iterating through the process allows earlier stages to be reworked to meet evolving needs.
- This approach can be applied to any of the generic process models, such as **incremental delivery** or **spiral development**.

Incremental Delivery

- **Incremental Delivery** means breaking down the development and delivery into smaller, manageable increments, where each increment delivers part of the required functionality.
- **User requirements** are prioritized, with high-priority requirements included in early increments.
- Once an increment's development begins, the requirements for that increment are **frozen**, though the requirements for later increments may evolve.

Key Takeaways

- The **process** plays a crucial role in determining the quality of the **product**. A weak process usually leads to poor outcomes, so care should be taken to ensure the process is sound and efficient.
- Both **incremental** and **spiral development** methods are essential when dealing with evolving requirements, as they allow for flexibility and continuous improvement throughout the development cycle.

Conclusions

- The different **life-cycle models**—each with its own strengths and weaknesses—should be selected based on:
    - The **organization** and its management style.
    - The **skills** of the development team.
    - The **nature** of the product being developed.
- **"Mix-and-match"** is suggested as the best approach, where different life-cycle models can be adapted to fit specific project needs.

Agile SDLC Overview:

- **Speed and Flexibility**: Agile methods prioritize speeding up or bypassing certain life cycle phases to rapidly adapt to changing requirements, especially in time-critical applications.
- **Less Formality**: Agile approaches are typically less formal and feature a reduced scope compared to traditional methods.
- **Adaptable to Changing Environments**: Agile is ideal for organizations that need to respond to change quickly, using disciplined methods.

Agility:

- **Effective Response to Change**: Agility enables effective responses to various types of changes, including team changes, technology advancements, and project modifications.
- **Key Features of Agility**:
    - **Rapid Delivery**: Focus on delivering working software quickly.
    - **Collaboration**: Customers are integrated into the development team.

- **Minimal Documentation**: Focus is on essential work products, keeping them lean.
- **Incremental Delivery**: Software is delivered in increments, allowing for faster customer feedback.

Principles of Agile Methods:

1. **Customer Involvement**: Customers are closely involved throughout development, prioritizing requirements and evaluating iterations.
2. **Incremental Delivery**: Software is developed in increments, with each iteration delivering a usable part of the system.
3. **People Over Process**: Agile focuses on the skills and adaptability of the development team rather than prescriptive processes.
4. **Embrace Change**: Systems are designed to accommodate changes in requirements.
5. **Simplicity**: Focus on simplicity in both the system and the development process.

Applicability of Agile Methods:

- **Product Development**: Agile is best suited for smaller, medium-sized products with clear customer involvement.
- **Custom System Development**: Agile works well when customers are committed to the process and external regulations are minimal.
- **Scaling Issues**: Scaling agile methods to large systems can be challenging due to their focus on small, tightly-knit teams.

Problems with Agile Methods:

- **Customer Engagement**: Keeping customers engaged can be difficult.
- **Team Compatibility**: Some team members may not be suited for the level of involvement required in Agile.
- **Prioritization**: Managing conflicting requirements from multiple stakeholders can be challenging.
- **Contractual Issues**: Agile's informal nature can create issues with contracts and requirement documents.

- **Adoption Challenges**: Organizations with established, formalized processes may struggle to adopt Agile methodologies.

Agile Methods and Software Maintenance:

- Agile is also useful for maintaining software, as it supports continuous changes and improvements after initial development.
- High-quality, well-structured code is crucial for maintainability in Agile.
- **Incremental Delivery**: Supports ongoing software evolution in response to customer feedback.

Plan-Driven vs. Agile Development:

- **Plan-Driven Development**: Involves detailed planning upfront, with structured stages and outputs.
- **Agile Development**: Involves overlapping phases, with outputs determined through negotiation during the development process.

Agile Methods in Comparison:

- **Agile**: Suitable for low criticality projects, where the team is small, and requirements change often.
- **Plan-Driven**: Suitable for high criticality projects, larger teams, and stable requirements.

Well-Known Agile Methodologies:

- **RAD** (Rapid Application Development)
- **RUP** (Rational Unified Process)
- **XP** (Extreme Programming)
- **Scrum**
- **TDD** (Test Driven Development)
- **FDD** (Feature Driven Development)
- **DSDM** (Dynamic Systems Development Method)
- **AUP** (Agile Unified Process)
- **Adaptive Software Development (ASD)**

- **Crystal Clear**

RAD (Rapid Application Development):

- **Phases of RAD**:
    1. **Requirements Planning**: Workshop to discuss business problems.
    2. **User Description**: Automated tools capture user data.
    3. **Construction**: Use of productivity tools inside a time-box.
    4. **Cutover**: Installation, testing, and training.
- **Strengths**: Faster cycle times, lower costs, and customer involvement.
- **Weaknesses**: Risk of incomplete closure, difficulty with legacy systems, and high dependence on committed developers.

When to Use RAD:

- When requirements are well-known and can be time-boxed.
- In scenarios where user involvement is crucial and the system can be modularized.

Extreme Programming (XP)

- **Target Audience**: Small-to-medium-sized teams working on projects with vague or rapidly changing requirements.
- **Key Features**:
    - **Coding Focus**: Coding is central, with code used to communicate and define the system's behavior.
    - **Iterative Development**: New versions are built rapidly and tested thoroughly.
    - **Customer Involvement**: Full-time engagement with the customer to define requirements and set priorities.
    - **Agile Principles**: Includes regular releases, simple design, frequent refactoring, and constant testing.

XP Practices

1. **Planning Game**: Prioritize user stories and tasks for the next release.

2. **Small Releases**: Quick releases to deliver functional systems frequently.
3. **Metaphor**: A simple shared story guides development.
4. **Simple Design**: Focus on simplicity, removing complexity as soon as it's detected.
5. **Testing**: Continuous unit tests and customer-written tests for features.
6. **Refactoring**: Continuous restructuring of the code to improve clarity and reduce duplication.
7. **Pair Programming**: Two programmers work together to write code.
8. **Collective Ownership**: Anyone can change any part of the codebase.
9. **Continuous Integration**: Frequent integration and testing of the codebase.
10. **40-Hour Workweek**: Avoid overtime to maintain productivity and code quality.
11. **On-Site Customer**: Customers are always available for feedback.
12. **Coding Standards**: Emphasize readability and clarity in code.

Scrum

- **Definition**: An agile methodology focusing on iterative, incremental development.
- **Key Roles**:
    - **Product Owner**: Manages the product backlog and makes decisions on priorities.
    - **Scrum Master**: Facilitates meetings, tracks progress, and removes obstacles for the team.
    - **Scrum Team**: Works on development tasks and organizes itself to achieve sprint goals.

Scrum Process

1. **Planning Phase**: Set project goals and architecture.
2. **Sprint Cycles**: Work is broken into fixed-length sprints (2-4 weeks), with regular reviews and adjustments.
3. **Daily Scrum**: Short meetings to track progress and resolve obstacles.
4. **Sprint Review**: Present completed work to stakeholders for feedback.

5. **Sprint Retrospective**: Reflect on the sprint process and make improvements.

Benefits of Scrum:

- **Improved Communication**: The whole team has visibility, leading to better collaboration.
- **Customer Feedback**: Regular deliveries help customers provide timely feedback.
- **Increased Trust**: Builds trust between customers and developers.
- **Continuous Improvement**: Scrum emphasizes iterative improvements.

Detailed Summary of Lecture 04: Requirements Engineering

Course: Software Engineering Concepts (CSC291)
Instructor: Dr. Tehseen Riaz Abbasi
Term: Fall 2024

Objectives:

- Define **requirements** and understand their importance in software engineering.
- Learn about the **Requirements Engineering (RE) process**, including elicitation, analysis, specification, validation, and management.
- Understand the structure of the **Software Requirements Specification (SRS)** and its importance.
- Differentiate between **functional** and **non-functional requirements**.
- Study the various **requirements gathering techniques** and methods to ensure clarity and accuracy.

What is a Requirement?

A **requirement** defines what a system should do or the constraints under which it operates. It can range from a high-level, abstract statement to a detailed functional specification. A requirement can serve two purposes:

1. **As a basis for a contract**: It must be detailed enough to define the scope of work.
2. **As a basis for a bid**: It should be broad enough to allow multiple solutions.

Agile Methods and Requirements:

- **Agile methods**, like **XP (Extreme Programming)**, claim that creating a fixed requirements document is inefficient due to frequent changes.
- Instead, requirements are gathered in an **incremental** manner and are expressed as **user stories**.
- While this is effective for business systems, it may not be suitable for critical systems that require extensive analysis.

What is Requirements Engineering (RE)?

Requirements Engineering is the process of discovering, analyzing, documenting, and validating the services and constraints of a system. It defines what the system should do and the conditions under which it operates.

The Requirements Engineering Process:

The **RE process** typically includes:

1. **Requirements Elicitation**: Gathering information from stakeholders.
2. **Requirements Analysis**: Organizing and structuring the gathered information.
3. **Requirements Specification**: Documenting the requirements in a structured format.
4. **Requirements Validation**: Ensuring the gathered requirements meet the needs of stakeholders.
5. **Requirements Management**: Continuously managing and updating requirements as the project progresses.

These activities can vary depending on the domain, team, and project complexity.

Stages of the Requirements Phase:

1. **Analysis**: Define the system's services, constraints, and goals through discussions with clients, customers, and users.
2. **Modeling**: Organize requirements in a structured manner for clarity and understanding.
3. **Definition and Communication**: Clearly document and share the requirements with all relevant stakeholders.

Requirements Elicitation and Analysis:

- **Elicitation** is the process of gathering requirements from stakeholders, including end-users, managers, domain experts, etc.

- **Analysis** involves organizing and prioritizing the gathered information to form a comprehensive and actionable list of requirements.

Steps include:

- **Requirements discovery**: Engaging stakeholders to gather information.
- **Requirements classification**: Grouping related requirements.
- **Requirements prioritization**: Ranking requirements based on importance and resolving conflicts.
- **Requirements specification**: Documenting the finalized requirements.

Problems in Requirements Analysis:

- Stakeholders may not know exactly what they want.
- Requirements can be expressed in varying, non-standard terms.
- Conflicting requirements may arise from different stakeholders.
- Organizational and political factors may influence system needs.
- Requirements may evolve over time due to new stakeholders or changing business conditions.

The Requirements Spiral:

The RE process is iterative, with activities such as discovery, classification, prioritization, and specification repeating throughout the project lifecycle. The spiral model allows for continuous refinement of requirements.

Requirements and Design:

While **requirements** define **what** the system should do, **design** describes **how** it will do it. However, in practice, requirements and design are often closely intertwined, as the architecture may influence the system's functionality and performance.

Requirements Specification:

The **requirements specification** involves documenting the system's user and system requirements in a clear and understandable manner. The specifications should be detailed enough for developers to implement but still comprehensible for non-technical stakeholders.

- **User Requirements**: High-level descriptions intended for non-technical users.
- **System Requirements**: More detailed and technical, often forming part of the system development contract.

Software Requirements Specification (SRS):

An **SRS** is a formal document that defines the system's behavior and interfaces. It covers functional, performance, and design constraints, and must be verifiable through methods like inspection, demonstration, analysis, or testing.

Natural Language Specification:

- Requirements are often written using **natural language** sentences with supporting diagrams or tables. This makes them understandable for users and customers.
- **Guidelines for writing requirements**:
    - Use a consistent format and language.
    - Avoid technical jargon.
    - Highlight key parts of the requirements.
    - Provide clear rationales for each requirement.

Problems with Natural Language:

- **Lack of clarity** and precision.
- **Confusion** between functional and non-functional requirements.
- **Amalgamation of requirements**, where multiple distinct requirements are expressed together.

Example Requirements:

- **Insulin Pump System**:
    - Measure blood sugar and deliver insulin every 10 minutes.
    - Run a self-test routine every minute to check for hardware/software issues.

Form-based Specifications:

A **form-based specification** includes:

- Function/entity definition.
- Description of inputs and outputs.
- Preconditions and postconditions.
- Any side effects or special actions required.

Key Concepts:

- **Requirements Elicitation**: Gathering requirements from stakeholders.
- **Requirements Analysis**: Organizing and classifying requirements.
- **Requirements Specification**: Documenting detailed requirements in a formal format.
- **Software Requirements Specification (SRS)**: A complete and precise document describing the system's behavior and interfaces.

Summary of Requirements Context:

In the context of Software Engineering, particularly in the **CSC291 Software Engineering Concepts** course, different types of requirements play crucial roles in ensuring the success of a system. These requirements include **User Requirements**, **System Requirements**, and **Domain Requirements**.

1. **User Requirements**:
    - These are typically written in natural language and may include diagrams. They describe the services the system should provide and the operational

constraints that are important for the customer. User requirements are written for the users, focusing on what they expect from the system.

2. **System Requirements**:
    - This is a more detailed document compared to user requirements. It describes the system's functions, services, and operational constraints. System requirements define exactly what the system should do and may be part of a contract between the client and contractor.
3. **Domain Requirements**:
    - These are specific to the system's operational domain. For instance, in a train control system, the system must account for environmental factors such as braking efficiency in different weather conditions. Domain requirements can be new functional requirements or constraints on existing ones. If domain requirements are not met, the system might fail to function.

Types of Requirements:

- **Functional Requirements**:
    - These describe what the system should do, focusing on the system's behavior in response to particular inputs or situations. They may also state what the system should **not** do.
    - **Example**: In a Medical Health Care Patient Management System (MHC-PMS), functional requirements might include the ability to search for patient appointments or generate a daily list of patients attending the clinic.
- **Non-Functional Requirements**:
    - These specify constraints on the system's functions, such as timing, performance, security, etc. Non-functional requirements define how the system should behave or the limits placed on its functionality.
    - **Examples**: This can include performance measures like response time, availability, or security features like user authentication.
    - **The FURPS Model** (Functionality, Usability, Reliability, Performance, and Supportability) is a common classification for non-functional requirements.

Non-Functional Requirement Classifications:

1. **Product Requirements**: Describe how the delivered product should behave (e.g., performance).
2. **Organizational Requirements**: Consequences of organizational policies (e.g., standards, processes).
3. **External Requirements**: External factors influencing the system (e.g., legislative requirements, interoperability).

Characteristics of Good Requirements:

- **Unambiguous**: Clear and easy to understand.
- **Testable**: Can be verified with specific tests.
- **Complete**: Describes all necessary facilities.
- **Consistent**: No contradictions within the requirements.
- **Understandable**: Easy for all stakeholders to interpret.
- **Feasible**: Realistic and achievable.
- **Independent**: Each requirement stands on its own.
- **Atomic**: Simple and precise.
- **Implementation-Free**: Abstract and independent of specific implementation.

Good vs. Bad Requirements:

- **Good requirements** are specific, measurable, and testable (e.g., "The system will respond within 2 seconds of user interaction").
- **Bad requirements** are vague and overly detailed (e.g., "The doorbell will be blue and round").

Imprecise, Incomplete, or Inconsistent Requirements:

- **Imprecision**: Problems arise when requirements are not clearly stated. For example, if a requirement says "search for a patient name," it's unclear whether it refers to searching across all clinics or within a single clinic.
- **Completeness and Consistency**: Requirements should be complete (covering all necessary functionality) and consistent (free of contradictions). However, achieving 100% completeness and consistency is often impractical.

Detailed Summary of Requirements Gathering Techniques

Requirements Discovery - Elicitation:
This is the process of gathering the necessary information to understand both the existing systems and user needs. This involves interactions with various stakeholders, from managers to external regulators. Information can come from documentation, system stakeholders, and similar system specifications.

Stakeholders in the MHC-PMS (Medical Health Center - Patient Management System):
Stakeholders are people who interact with or are impacted by the system. For MHC-PMS, the key stakeholders include:

- **Patients** who have their information recorded in the system.
- **Doctors** responsible for assessing and treating patients.
- **Nurses** who coordinate consultations and administer treatments.
- **Medical receptionists** who manage patient appointments.
- **IT staff** who maintain and install the system.

Techniques for Eliciting Requirements:

There are several methods for gathering requirements from customers, such as:

1. **Surveys/Questionnaires:**
   A set of questions designed to gather specific information. These can range from simple Yes/No answers to more complex multiple-choice options. Tools like **SurveyMonkey** (cloud-based) and **LimeSurvey** (offline) are often used for creating and distributing these surveys.
2. **Focus Groups and Workshops:**
   Focus groups involve gathering a group of stakeholders to discuss the issues and requirements.
    - **Focus Group:** A facilitator guides the discussion with a pre-prepared list of questions based on the participants' experience. This helps identify the system's requirements.
3. **Naturalistic Observation:**
   This involves observing stakeholders in their regular work environment to understand how they perform tasks. It can give valuable insights into their tasks and how they interact with the system. However, it requires a significant time commitment and may produce a lot of data to analyze. Bias in observations can be an issue, as people might behave differently when being watched.

4. **Document Analysis:**
   Studying existing documentation, such as procedures, business forms, reports, and system descriptions, helps to understand how tasks are performed and what regulations need to be followed. This provides valuable data for requirements gathering.

5. **Interviews:**
   In interviews, the analyst asks a set of questions to stakeholders. Interviews can be:

   - **Closed:** With a predetermined list of questions.
   - **Open:** Where the interviewer explores various issues with stakeholders. Effective interviews involve being open-minded, asking questions that prompt deeper discussions, and being willing to listen to the stakeholders' needs.

6. **Ethnography:**
   Ethnography involves spending time in the environment where the system will be used to understand the actual tasks and processes that occur. This helps to uncover implicit system requirements that are based on how people work in real life, rather than just formal processes. It is particularly useful for understanding the context in which a system operates and the cooperation needed among team members.

7. **Prototyping:**
   Prototyping involves building an early version of the system to quickly develop concrete specifications and gain feedback from users. This iterative process helps refine system requirements based on user input and feedback.

8. **Brainstorming:**
   A group activity where people generate ideas and solutions without judgment. Brainstorming sessions can be either individual or group-based and are useful for coming up with new concepts or features for a system.

9. **Task Descriptions, Scenarios, and Use Cases:**

   - **Scenarios** are informal narrative stories that describe how users will interact with the system. They help to provide a more detailed outline of the requirements.
   - **Use Cases** show how users interact with the system in detail. They help to define the system's functionality and provide a deeper understanding of its interactions.

Summary of Key Techniques:

- **Surveys/Questionnaires** are useful for gathering structured feedback.
- **Focus Groups** involve group discussions to gain diverse insights.
- **Naturalistic Observation** allows analysts to see users' actual workflows.
- **Document Analysis** provides insights from existing procedures and reports.
- **Interviews** offer in-depth perspectives from stakeholders.
- **Ethnography** helps in understanding users' behaviors and interactions in context.
- **Prototyping** allows users to interact with early system versions to refine requirements.
- **Brainstorming** generates creative ideas in group settings.
- **Scenarios and Use Cases** define user interactions with the system in detail.

cenarios:

- A scenario is a real-life example of how a system is used.
- It describes:
    - The starting situation.
    - The normal flow of events.
    - Possible problems that could occur.
    - Any other ongoing activities.
    - The state of the system after the scenario finishes.

Use Cases:

- Use cases identify actors (users or systems) and the interactions between them and the system.
- They describe all possible interactions with the system and help in understanding its behavior.
- Use cases are often represented with diagrams and can be detailed using sequence diagrams.

- They include both successful interactions (main flow) and failure scenarios (alternative flows).
- Use cases do not focus on user interface design or implementation details unless essential for achieving the goal.

Use Case Structure:

- A use case includes:
    - A **trigger** that starts the interaction.
    - **Preconditions** that must be true before the use case begins.
    - A **normal flow** that describes the main interaction.
    - **Alternative flows** for unexpected situations.
    - **Exceptions** that deal with error conditions.
    - **Postconditions** that describe the state after the interaction.
    - **Assumptions** and **business rules** that may affect the interaction.

Use Case Example:

- For an "ATM Withdrawal" use case, the normal flow could involve inserting a card, entering a PIN, selecting the amount, and completing the transaction. Alternative flows might involve entering the wrong PIN or insufficient balance, with each step clearly described.

Tabular Format for Writing Use Cases:

- The format includes:
    - Use Case ID, Name, Actors, Description, Trigger, Preconditions, Normal Flow, Alternative Flows, Exceptions, Postconditions, Assumptions, and Notes.

Functional Requirements:

- These describe the system's behavior from a functional perspective, such as allowing users to edit applications or validate email addresses. They include the requirement’s description, source, rationale, dependencies, and priority.

Requirements Validation and Management Summary

Requirements Validation

- **Definition**: Requirements validation ensures that all software requirements are stated clearly, identifying any inconsistencies, omissions, and errors, and ensuring conformity to established standards for process, project, and product.
- **Importance**: Requirements errors are costly; fixing them after delivery can be up to 100 times more expensive than fixing implementation errors.
- **Main Concerns**:
    - **Validity**: Does the system provide the functions needed by the customer?
    - **Consistency**: Are there any conflicting requirements?
    - **Completeness**: Are all necessary functions included?
    - **Realism**: Can the requirements be implemented with available resources?
    - **Verifiability**: Can the requirements be tested?

Requirements Validation Techniques

- **Requirements Reviews**: Group reviews by both developers and customers to check for errors and inconsistencies.
- **Prototyping**: Using an executable model to validate requirements.
- **Test-case Generation**: Creating test cases to verify the requirements.

Review Checks

- **Verifiability**: Is the requirement realistically testable?
- **Comprehensibility**: Is the requirement well-understood?
- **Traceability**: Can the requirement be traced back to its origin?
- **Adaptability**: Can the requirement be changed easily?

Requirements Management

- **Definition**: Managing changes in requirements throughout the development process.
- **Challenges**: New requirements emerge as the system is developed and after it goes live.
- **Key Activities**:

- Track individual requirements and their interdependencies.
- Establish a process for proposing and implementing requirement changes.
- Address changes in business and technical environments.

Changing Requirements

- **Factors**:
  - Changes in business or technical environment.
  - Conflicts between business and end-user needs.
  - Legislation or regulation changes.
- **Types of Requirements**:
  - **Enduring**: Stable, e.g., essential components like doctors in a hospital system.
  - **Volatile**: Subject to change, e.g., healthcare policies affecting a hospital system.

Requirements Change Management

- **Process**:
  1. **Problem analysis**: Understand the problem or change request.
  2. **Change analysis and costing**: Assess the impact of changes.
  3. **Change implementation**: Modify the requirements document and other relevant documentation.

Traceability

- **Importance**: Traceability links requirements to their sources and the system design, enabling the assessment of the impact of proposed changes.
- **Types**:
  - **Source traceability**: Link from requirements to stakeholders.
  - **Requirements traceability**: Links between dependent requirements.
  - **Design traceability**: Links from requirements to design.

Tools and Techniques

- **CASE Tool Support**: Tools that support requirements storage, change management, and traceability management.
- **Requirements Change Management Stages**:
    - **Problem analysis**
    - **Change analysis and costing**
    - **Change implementation**

Home Task: Use Case Diagram for Train Ticket Distributor

Create a use case diagram for a ticket distribution system with the following actors and use cases:

- **Actors**:
    1. **Traveler**: Purchases different types of tickets.
    2. **Central Computer System**: Maintains the reference database for ticket tariffs.
- **Use Cases**:
    - **BuyOneWayTicket**
    - **BuyWeeklyCard**
    - **BuyMonthlyCard**
    - **UpdateTariff**
- **Exceptional Cases**:
    - **TimeOut**: Traveler takes too long to insert the correct amount.
    - **TransactionAborted**: Traveler cancels the transaction before completion.
    - **DistributorOutOfChange**: Distributor runs out of change.
    - **DistributorOutOfPaper**: Distributor runs out of paper.

Let's break down the **use case diagram** for the ticket distribution system. Here's how we'll proceed step-by-step:

1. Identify the Actors:

- **Traveler**: The user who interacts with the system to buy tickets.
- **Central Computer System**: Manages the tariff reference database and updates the ticket prices.

2. Identify the Use Cases:

- **BuyOneWayTicket**: Traveler buys a one-way ticket.
- **BuyWeeklyCard**: Traveler buys a weekly travel card.
- **BuyMonthlyCard**: Traveler buys a monthly travel card.
- **UpdateTariff**: Central system updates the pricing for tickets.

3. Exceptional Use Cases:

- **TimeOut**: Traveler takes too long to insert the correct amount of money.
- **TransactionAborted**: Traveler cancels the transaction before completion.
- **DistributorOutOfChange**: The distributor runs out of change for the traveler.
- **DistributorOutOfPaper**: The distributor runs out of ticket paper.

4. Use Case Relationships:

- **Include Relationship**:
    - The BuyTicket use case may be included in the BuyOneWayTicket, BuyWeeklyCard, and BuyMonthlyCard since buying tickets would generally involve the same steps.
- **Extend Relationship**:
    - **TimeOut**, **TransactionAborted**, **DistributorOutOfChange**, and **DistributorOutOfPaper** are exceptional cases and would extend the BuyTicket use case, as they happen under specific conditions during the ticket buying process.

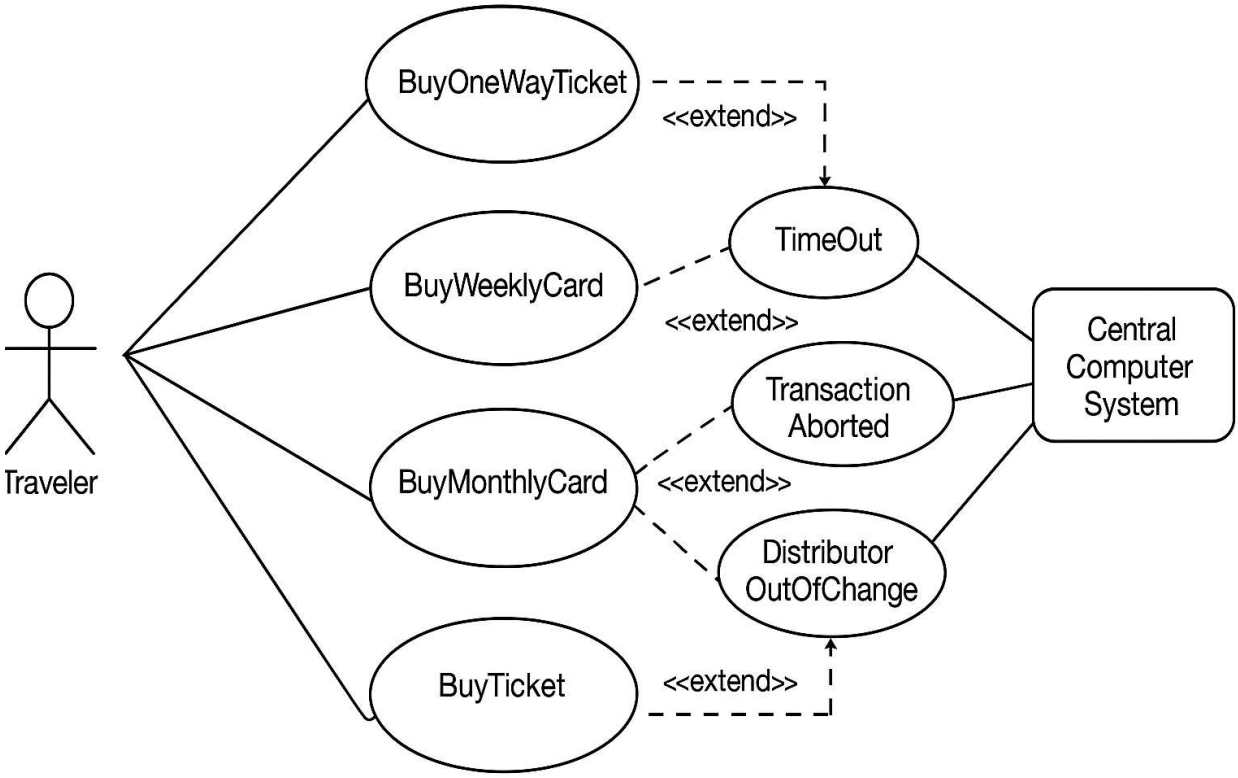
5. Creating the Use Case Diagram:

Here's a visual outline of how the use case diagram would look:

1. **Actors**:

- **Traveler** will be linked to the BuyOneWayTicket, BuyWeeklyCard, and BuyMonthlyCard use cases.
- **Central Computer System** will be linked to the UpdateTariff use case.

2. **Use Cases**:
   - BuyOneWayTicket, BuyWeeklyCard, BuyMonthlyCard, UpdateTariff.
   - **Exceptions** like TimeOut, TransactionAborted, DistributorOutOfChange, and DistributorOutOfPaper extend the BuyTicket use case.



Requirements (NFRs) in Software Quality Engineering, specifically focusing on **McCall's Quality Model (1977)**.

In simple terms, while functional requirements describe *what* a software system should do, non-functional requirements describe *how well* it should do it. McCall's model categorizes these "how well" aspects into several key quality factors:

Here's a breakdown of the NFRs discussed:

1. Correctness:

- This focuses on the **accuracy and completeness** of the software's outputs.
- It considers factors like:
    - **Accuracy of output:** How precise the information is (e.g., probability of inaccurate inventory not exceeding 1%).
    - **Completeness of output:** Ensuring all necessary information is present (e.g., probability of missing phone number or address in a member's record not exceeding 1%).
    - **Up-to-datedness of information:** How current the information is and how frequently it's updated (e.g., payment information updated within one working day with 99.5% probability).
    - **Response time:** How quickly the system provides requested information or reacts to events (e.g., inventory query response time under 2 seconds, valve reaction time under 10 seconds).
    - **Coding and documentation standards:** Following established rules for writing and documenting the software.

2. Reliability:

- This deals with the software's ability to operate **without failure**.
- It sets limits on:
    - **Failure rate:** How often the system is allowed to fail (e.g., heart-monitoring unit failure less than once in 20 years).
    - **Downtime:** The maximum allowed time the system is unavailable (e.g., bank system failing no more than once per 12 months during office hours).
    - **Recovery time:** How quickly the system can be restored after a failure (e.g., probability of bank service recovery taking more than 10 minutes being less than 0.5%).

3. Efficiency:

- This concerns the **hardware resources** the software needs to perform its functions effectively.

- Key resources include:
    - **Processing capabilities:** How much processing power the system requires (e.g., measured in MIPS).
    - **Data storage:** The amount of memory and disk space needed (e.g., measured in GBs, TBs).
    - **Data communication:** The bandwidth required for data transfer (e.g., measured in MBPSs, GBPSs).
- It also considers the **power consumption** and the time between recharges for portable devices.
- The examples illustrate comparing different system bids based on their hardware resource usage.

4. Integrity:

- This focuses on **software security** and preventing unauthorized access and modifications.
- It involves defining different levels of access (e.g., read-only vs. write access) for different users.
- The example of the municipality's GIS highlights restricting citizens' access to viewing and copying information but preventing changes to maps.

5. Usability:

- This relates to how **easy it is for users to learn and operate** the software.
- It considers:
    - **Operation usability:** User productivity, measured by the number of tasks completed per unit of time (e.g., a helpdesk staff member handling at least 60 calls a day).
    - **Training usability:** The time and effort required to train new users (e.g., training a new employee taking no more than 2 days).

6. Maintainability:

- This defines how **easy it is to modify, correct, and adapt** the software after it's been developed.

- It often relates to the software's internal structure, documentation, and coding standards (e.g., limiting the size of software modules, adhering to company coding guidelines).

7. Flexibility:

- This refers to the software's ability to **adapt to different needs and environments** without requiring significant changes.
- It supports:
    - **Adaptive maintenance:** Adjusting the software for different customers or situations (e.g., a teacher support software working for different subjects and school levels).
    - **Perfective maintenance:** Making improvements and adding new features to enhance the software or adapt to new technologies or business needs (e.g., non-professionals creating new report types in the teacher support software).

8. Testability:

- This concerns how **easy it is to test** the software to ensure it works correctly.
- It includes:
    - Features that aid testing (e.g., providing predefined test data and expected results, log files).
    - Automatic diagnostics performed by the software itself (e.g., checking if all components are working before operation, detecting faults).

9. Portability:

- This describes the software's ability to **run in different environments** (e.g., different operating systems, hardware).
- The example mentions the requirement for a software package designed for Windows to be easily transferred to Linux.

10. Reusability:

- This focuses on the ability to **use existing software components** in new projects or to design new components that can be used in future projects.

- Reusing software can save development time and resources and potentially lead to higher quality due to prior testing.
- The example of the hotel swimming pool software highlights designing modules that can be reused in the spa's future system.

11. Interoperability:

- This deals with the software's ability to **work with other software systems or hardware**.
- It often involves defining interfaces and data exchange formats (e.g., a medical lab equipment's output following a standard data structure to be used by lab information systems, the lab system interfacing with a clinic system).

In summary, this presentation introduces the crucial concept of Non-Functional Requirements in software engineering using McCall's Quality Model as a framework. It explains various quality attributes beyond just what the software does, focusing on how well it performs in terms of correctness, reliability, efficiency, security, usability, maintainability, adaptability, testability, portability, reusability, and its ability to work with other systems. Understanding and defining these NFRs are essential for building high-quality software that meets user needs and performs effectively in its intended environment.